Manuel Utilisateur

GL03



CAMATCHY Alexander
CANAGUY Vincent
GRAHAM Martha
GUINET Aster
LAISNÉ Matthew
MATER Hugo
NGONGANG Gaston
THOMAZIC Valentin
WENDELS Jérémy

Table des matières

1. Limitations du compilateur _____ -3-
2. Options du compilateur _____ -3-
3. Messages d'erreur _____ -4-
   3.1. Erreurs de syntaxe _____ -4-
   3.2. Erreurs de syntaxe contextuelle _____ -5-
   3.3. Erreurs d'exécution du code assembleur _____ -7-

1. Limitations du compilateur

Notre compilateur est limité à la compilation du Deca objet.

2. Options du compilateur

Les options implémentés pour le compilateur sont :

- **-p (parse)** : arrête decac après l'étape de construction de l'arbre, et affiche la décompilation de ce dernier (i.e. s'il n'y a qu'un fichier source à compiler, la sortie doit être un programme deca syntaxiquement correct)
- **-v (verification)** : arrête decac après l'étape de vérifications (ne produit aucune sortie en l'absence d'erreur)
- **-r X (registers)** : limite les registres banalisés disponibles à R0 … R{X-1}, avec 4 <= X <= 16. Si l'utilisateur entre un X < 4 (respectivement un X > 16), X est forcé à la borne inférieure 4 (respectivement la borne supérieure 16)
- **-d (debug)** : active les traces de debug. Répéter l'option plusieurs fois pour avoir plus de traces
- **-P (parallel)** : s'il y a plusieurs fichiers sources, lance la compilation des fichiers en parallèle (pour accélérer la compilation)
- **-n (no check)** : désactive certains tests devant être soulevé à la compilation

3. Messages d'erreur

Les messages d'erreur ont le format suivant : « Emplacement dans le code - type d'erreur - précisions sur l'erreur ».

3.1. Erreurs de syntaxe

Type	**Description**	**Exemple de code provoquant l'erreur**	**Exemple de message d'erreur**

Erreur de syntaxe	La syntaxe de deca n'est pas respecté	`{`
		`  if;`
		`  int x;`
		`  if(!x;`
		`}`
			mismatched input ‹) › expecting {‹ false ›, ‹ new ›, ‹ null ›, ‹ readInt ›, ‹ readFloat ›, ‹ this ›, ‹ true ›, ‹ - ›, ‹ (‹ , ‹ ! ›, INT, IDENT, FLOAT, STRING}

3.2. Erreurs de syntaxe contextuelle

Type	Description	Exemple de code provoquant l'erreur	Exemple de message d'erreur
IncompatibleTypes	Les types de l'opération sont incompatibles	`{ int x; if(!x); }`	Incompatible Types: RValue must have the same type or subtype as LValue: boolean instead of int
DoubleDefException	La variable est définie plus d'une fois	`{ int x; float x; }`	variable `x` is already defined
NullDefinition	Utilisation d'un identificateur qui n'a pas été défini	`{ if(x); }`	variable `x` is not defined
UndefinedType	Définition d'un identificateur avec un type non défini	`{ double x; }`	type `double` is not defined
Illegal operand	Un opérateur est utilisé de la mauvaise façon	`{ true + true }`	Illegal operand ‹ + › between boolean and boolean
MethodCall	Exception en cas de mauvais appel de fonction	`class A{ int b(int x){} } { A test = new A(); boolean o = true; test.b(o); }`	MethodCall: Cannot call method b with parameters boolean o instead of int x

Type	**Description**	**Exemple de code provoquant l'erreur**	**Exemple de message d'erreur**
ReturnError	Exception en cas d'absence de return	`class A{ int b(){} }`	ReturnError: Missing return statement in non-void method
FieldAccesException	Exception en cas de mauvais accés à un champ protégé	`class A{ protected int x; } { int y = new A().x; }`	Protected field x is not accessible from main

3.3. Erreurs d’exécution du code assembleur

Type	Description	Exemple de code provoquant l’erreur	Exemple de message d’erreur
InputOutputError	Lecture ou écriture d’un type incompatible	`{ RINT }` ;l’utilisateur entre un `string`	expected `int` but got `string` instead
StackOverflow	Accès mémoire non autorisé, dépassement de la pile mémoire		
IllegalOperation	Division par 0, modulo 0	`{ int x = 0 int y = 2; y = y / x; }`	division of `y` by 0